#Semaphore signalling #

```
#include <includes.h>

#include "func.h"


void InitBoard(void);


/*--------------- AAPLICATION STACKS ---------*/

static  OS_STK  AppTask0stk[256];       /* Create the required number of stacks need for every child task*/

static  OS_STK  AppTask1stk[256];


/*--------------- CHILDERN TRASKS --------------*/

static  void  AppTask0(void *p_arg);

static  void  AppTask1(void *p_arg);


OS_EVENT *Sem;                          /* Semaphore  */


int  main (void)

{

   InitBoard();

   OSInit();                          /* Initialize "uC/OS-II, The Real-Time Kernel"            */


   Sem = OSSemCreate(0);               /* Create a Semaphore */


   OSTaskCreate(AppTask0,0,&AppTask0stk[256 - 1],4);

   OSTaskCreate(AppTask1,0,&AppTask1stk[256 - 1],5);


   OSStart();                                  /* Start multitasking (i.e. give control to uC/OS-II)
*/
```

```
}

static  void  AppTask0 (void *p_arg)
{
  p_arg = p_arg;                          /*Just to avoid compiler Warning          */

   while(DEF_TRUE)
   {
   /* User Code Here */
      printf("TASK0: Press any key on keypad to release Semaphore\n\r");
      OSTimeDlyHMSM(0,0,2,0);
      while(!(GetKey()));                  /*  Wait till a key is pressed  */
      OSSemPost(Sem);                      /*  Signal Semaphore    */
      printf("TASK0: Semaphore released.... \n\r");
      OSTimeDlyHMSM(0,0,5,0);
   }
}

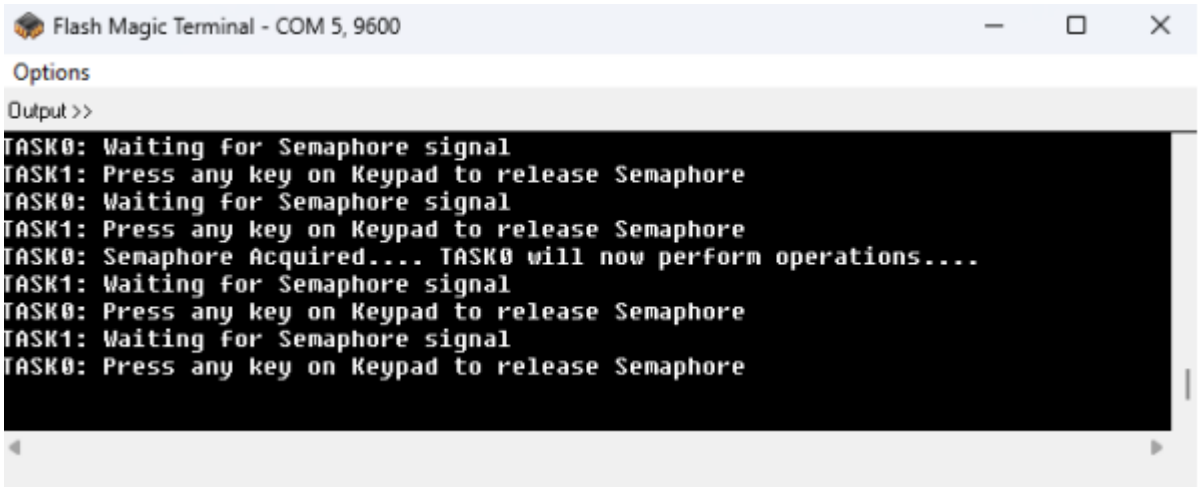

static  void  AppTask1 (void *p_arg)
{
unsigned char err;
  p_arg = p_arg;                          /* Just to avoid compiler Warning        */

  while(DEF_TRUE)
  {
    /* User Code Here */
      printf("TASK1: Waiting for Semaphore signal\n\r");
```

```
        OSSemPend(Sem,200,&err);                        /*  Wait on Semaphore   */
        while(err != OS_NO_ERR);
        printf("TASK1: Semaphore Acquired.... TASK1 will now perform operations....\n\r\r\n");
        OSTimeDlyHMSM(0,0,3,0);


    }
}

void InitBoard(void)
{
    BSP_IntDisAll();                /* Disable all interrupts until we are ready to accept them */
    BSP_Init();
    InitLCD();                  /* Initialize LCD   */
    kbdInit();                  /* Initialize Keyboard  */
    ADCInit();                  /* Initialize ADC   */
    LEDInit();                  /* Initialize LED   */
    UartInit(9600);                 /* Initialise the UART*/
}
```



#semaphore Symchronise#

```
#include <includes.h>
#include "func.h"

void InitBoard(void);

/*--------------- AAPLICATION STACKS ---------*/
static  OS_STK  AppTask0stk[256];      /* Create the required number of stacks need for every child task*/
static  OS_STK  AppTask1stk[256];

/*--------------- CHILDERN TRASKS --------------*/
static  void  AppTask0(void *p_arg);
static  void  AppTask1(void *p_arg);

OS_EVENT *Sem;                          /* Semaphore  */

int  main (void)
{
   InitBoard();
   OSInit();                          /* Initialize "uC/OS-II, The Real-Time Kernel"            */


   Sem = OSSemCreate(0);               /* Create a Semaphore */

   OSTaskCreate(AppTask0,0,&AppTask0stk[256 - 1],4 );
   OSTaskCreate(AppTask1,0,&AppTask1stk[256 - 1],5 );
```

```
   OSStart();                                  /* Start multitasking (i.e. give control to uC/OS-II)
*/
}


static  void  AppTask0 (void *p_arg)
{
  unsigned char err;
   p_arg = p_arg;                            /*Just to avoid compiler Warning           */


   while(DEF_TRUE)
   {
    /* User Code Here */
      do                                /*   Wait till a key is pressed  */
      {
         printf("TASK0: Press any key on Keypad to release Semaphore\n\r");
         OSTimeDlyHMSM(0,0,2,0);
      }while(!(GetKey()));
      OSSemPost(Sem);                           /*   Signal Semaphore    */
      OSTimeDlyHMSM(0,0,0,200);
      do
      {
         printf("TASK0: Waiting for Semaphore signal\n\r");
         OSSemPend(Sem,200,&err);
      }while(err != OS_NO_ERR);                     /*   Wait on Semaphore   */
      printf("TASK0: Semaphore Acquired.... TASK0 will now perform operations....\n\r");
      OSTimeDlyHMSM(0,0,1,0);
   }
```

```
}



static  void  AppTask1 (void *p_arg)
{
unsigned char err;
   p_arg = p_arg;                                /* Just to avoid compiler Warning         */

   while(DEF_TRUE)
   {
    /* User Code Here */
      do
      {
         printf("TASK1: Waiting for Semaphore signal\n\r");
         OSSemPend(Sem,200,&err);                          /* Wait on Semaphore   */
      }while(err != OS_NO_ERR);
      printf("TASK1: Semaphore Acquired.... TASK1 will now perform operations....\n\r");
      OSTimeDlyHMSM(0,0,1,0);
      do                                    /* Wait till a key pressed */
      {
         OSTimeDlyHMSM(0,0,2,0);
         printf("TASK1: Press any key on Keypad to release Semaphore\n\r");
      }while(!(GetKey()));
      OSSemPost(Sem);                                /* Signal Semaphore    */
      OSTimeDlyHMSM(0,0,0,200);

   }
}
```

```
void InitBoard(void)
{
   BSP_IntDisAll();                  /* Disable all interrupts until we are ready to accept them */
   BSP_Init();
   InitLCD();                        /* Initialize LCD   */
   kbdInit();                        /* Initialize Keyboard  */
   ADCInit();                        /* Initialize ADC   */
   LEDInit();                        /* Initialize LED   */
   UartInit(9600);                   /* Initialise the UART*/
}
```

